

CarePath — เอกสารออกแบบระบบและขั้นตอนการพัฒนา

Hackathon 977-121 Module: Website Design and Development · วิทยาลัยการคอมพิวเตอร์ มหาวิทยาลัยสงขลานครินทร์ วิทยาเขตภูเก็ต

ระบบนำทางในโรงพยาบาลและติดตามขั้นตอนการรักษา · โจทย์ WIL โรงพยาบาลวชิระภูเก็ต

วันที่ 1: เสาร์ที่ 19 กันยายน 2569 · วันที่ 2: อาทิตย์ที่ 20 กันยายน 2569

เอกสารฉบับนี้ครอบคลุมอะไร — การออกแบบระบบทั้งหมด (สถาปัตยกรรม · ฐานข้อมูล · API · อัลกอริทึม · UX) และขั้นตอนการพัฒนาตามลำดับเวลาจริงทั้งสองวัน พร้อมเหตุผลของทุกการตัดสินใจ

ผังอาคาร ชั้น แผนก เส้นทางเดิน และผู้ป่วยทั้งหมดเป็นชุดสมมติที่สร้างขึ้นเพื่อสาธิตกลไก ยังไม่ใช่ผังจริงของโรงพยาบาล ไม่มีข้อมูลบุคคลจริงในระบบ

สารบัญ

1. โจทย์และขอบเขต

2. สถาปัตยกรรมระบบ

3. การออกแบบฐานข้อมูล

4. การนอร์มัลไลเซชัน

5. API Contract

6. อัลกอริทึมที่เป็นหัวใจ

7. การออกแบบ UX/UI

8. ความปลอดภัยและความเป็นส่วนตัว

9. ขั้นตอนการพัฒนา วันที่ 1

10. ขั้นตอนการพัฒนา วันที่ 2

11. การทดสอบ

12. ข้อจำกัดและงานต่อ

1. โจทย์และขอบเขต

1.1 ปัญหาจากหน่วยงาน

ผู้ป่วยนอกของโรงพยาบาลขนาดใหญ่ต้องผ่านหลายจุดบริการในวัน — เวชระเบียน คัดกรอง เจาะเลือด รอผล พบแพทย์ ชำระเงิน รับยา — โดยแต่ละจุดอยู่คนละอาคารคนละชั้น ปัญหาที่เกิดขึ้นทุกวันคือ

- ผู้ป่วยไม่รู้ว่ **ตอนนี้ต้องไปที่ไหนต่อ** เพราะโบบนบอกแค่รายการ ไม่ได้บอกลำดับหรือเส้นทาง
- ป้ายบอกทางออกแบบสำหรับคนที่รู้ผังอยู่แล้ว ผู้สูงอายุและผู้ใช้รถเข็นเดินผิดทางแล้วต้องย้อนกลับ
- เจ้าหน้าที่ต้องตอบคำถามเดิมซ้ำทั้งวัน แทนที่จะได้ทำงานของตัวเอง
- แรงงานต่างชาติและนักท่องเที่ยวอ่านป้ายภาษาไทยไม่ออก

1.2 สิ่งทีระบบต้องทำ (Functional)

#	ความต้องการ	ผู้ใช้
F1	เปิดการมาโรงพยาบาลหนึ่งครั้ง (visit) จากแม่แบบขั้นตอน พร้อมออกบัตรคิวที่มี QR	เวชระเบียน
F2	บอกผู้ป่วยว่าขั้นถัดไปคืออะไร อยู่ที่ไหน และเดินไปยังไงทีละข้อ	ผู้ป่วย
F3	คิดเส้นทางตามความต้องการพิเศษ — รถเข็นต้องไม่เจอบันได	ผู้ป่วย
F4	เจ้าหน้าที่จุดบริการเปลี่ยนสถานะขั้นตอนด้วยปุ่มเดียว แล้วหน้าผู้ป่วยเปลี่ยนตามเอง	จุดบริการ
F5	แทรกขั้นตอนระหว่างวันได้ (แพทย์สั่งตรวจเพิ่ม) แล้วจัดลำดับใหม่ให้ถูกต้อง	จุดบริการ
F6	ปิดเส้นทางได้ (ลัพท์ซ่อม) แล้วคำนวณทางใหม่และแจ้งผู้ป่วยที่กระทบ	ผู้ดูแลระบบ
F7	ดูผลกระทบก่อนตัดสินใจปิดเส้นทาง — ใครเดินเพิ่มเท่าไร ใครไปไม่ถึง	ผู้ดูแลระบบ
F8	ค้นผู้ป่วยด้วยเลขคิวเมื่อเดินหลงมาถาม แล้วยืนยันตำแหน่งให้	ประชาสัมพันธ์

1.3 สิ่งทีระบบต้องเป็น (Non-functional)

#	ข้อกำหนด	วิธีทำให้เป็นจริง
N1	ผู้ป่วยไม่ต้องติดตั้งแอปและไม่ต้องสมัครสมาชิก	เว็บล้วน · เข้าด้วย token ในลิงก์ของบัตรคิว
N2	เจ้าหน้าที่ต้องเสียเวลาเพิ่มไม่เกิน 2 วินาทีต่อผู้ป่วยหนึ่งคน	ปุ่มเดียวต่อหนึ่งสถานะ ไม่มีฟอร์มให้กรอก
N3	รองรับผู้อ่านไทยไม่ออก	5 ภาษา — ไทย อังกฤษ จีน เมียนมา รัสเซีย
N4	รองรับผู้ที่ไม่มีมือถือ	บัตรกระดาษพิมพ์คำสั่งเดินละเอียดขึ้น · เจ้าหน้าที่ยืนยันตำแหน่งแทน
N5	ผังจริงของโรงพยาบาลต้องนำเข้าได้โดยไม่ต้องแก้โค้ด	ผังทั้งหมดอยู่ใน CSV และฐานข้อมูล ไม่มีที่ไหน hard-code
N6	ข้อมูลผู้ป่วยต้องตรวจสอบย้อนหลังได้และแก้ไขไม่ได้	ตาราง audit append-only · <code>REVOKE UPDATE, DELETE</code>
N7	ฐานข้อมูลต้องเป็น PostgreSQL และอยู่ในรูป 3NF	17 ตาราง · ทุก query เป็น parameterized

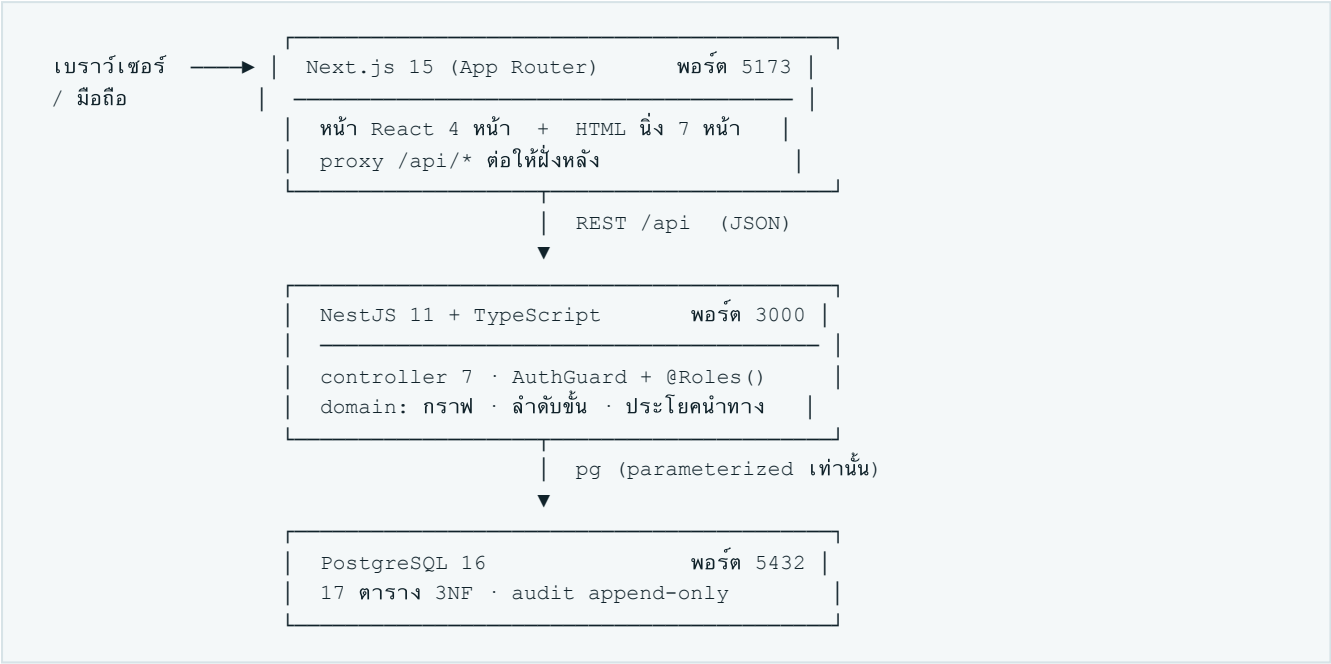
1.4 ขอบเขต MVP

อยู่ในขอบเขต — การมาโรงพยาบาลของผู้ป่วยนอกหนึ่งวัน ตั้งแต่วันบัตรถึงรับยา · บทบาทผู้ใช้ 4 แบบ (เวชระเบียน จุดบริการ ผู้ดูแลระบบ ผู้บริหาร) · ผังสมมุติ 3 อาคาร 50 จุด

นอกขอบเขต — ผู้ป่วยใน · การนัดหมายล่วงหน้า · การเชื่อมกับ HIS ของโรงพยาบาล · การชำระเงินจริง · แอปมือถือ

2. สถาปัตยกรรมระบบ

2.1 ภาพรวมสามชั้น



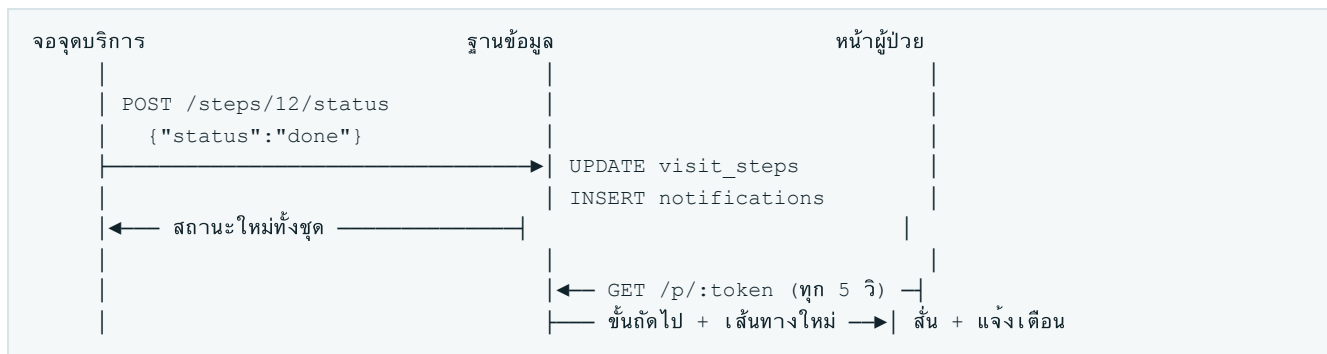
ทั้งสามชั้นเป็นคนละ container ใน `docker-compose.yml` เบราว์เซอร์เห็นพอร์ตเดียวคือ 5173 Next.js เป็นคน proxy `/api/` ต่อให้ ไม่มีหน้าไหนต่อฐานข้อมูลเอง และไม่มีรหัสผ่านหรือ สิทธิ์ผู้ใช้อยู่ในโค้ดฝั่งหน้าเว็บเลย

2.2 เหตุผลของการแยกชั้น

การตัดสินใจ	เหตุผล
Frontend กับ Backend แยกกันจริง คุยผ่าน REST เท่านั้น	เป็นข้อกำหนดของโจทย์ และทำให้เปลี่ยนฝั่งใดฝั่งหนึ่งได้โดยไม่กระทบอีกฝั่ง — พิสูจน์แล้ววันที่ 2 ตอน เปลี่ยนทั้งสองฝั่งคนละรอบ
Next.js เป็นคน proxy <code>/api</code> แทนที่จะให้เบราว์เซอร์โดยตรง	เบราว์เซอร์เห็น origin เดียว จึงไม่ต้องตั้ง CORS และไม่ต้องเปิดพอร์ต API ออกสู่ภายนอก
ตรรกะโดเมนเป็นฟังก์ชันล้วน แยกจาก controller และฐานข้อมูล	ทดสอบอัลกอริทึมได้โดยไม่ต้องมีฐานข้อมูล — ทЕСТ 24 กรณีรันเสร็จใน 9 วินาที
สถานะทั้งหมดอยู่ในฐานข้อมูล ไม่มีใน session ของเซิร์ฟเวอร์	จอบุคลากรกับหน้าผู้ป่วยอ่านความจริงชุดเดียวกัน จึงไม่มีทางเห็นไม่ตรงกัน

2.3 การชิงกระหว่างสองหน้าจอ

หัวใจของระบบคือ เจ้าหน้าที่กดปุ่มแล้วหน้าผู้ป่วยเปลี่ยนเอง ทำด้วยวิธีที่ง่ายที่สุดที่ยังตอบโจทย์



เลือก polling แทน WebSocket เพราะ 5 วินาทีเพียงพอสำหรับงานที่คนเดินเป็นนาทีก และไม่ต้องดูแล connection state ซึ่งเป็นความซับซ้อนที่ไม่คุ้มในขอบเขตนี้

3. การออกแบบฐานข้อมูล

3.1 แนวคิดหลัก — ผังโรงพยาบาลคือกราฟ

การตัดสินใจที่สำคัญที่สุดของโครงการคือ เก็บผังเป็นกราฟ ไม่ใช่รูปภาพ ทุกจุดในโรงพยาบาลเป็นโหนด (nodes) ทุกทางเดิน ลิฟต์ บันได ทางลาด ทางเชื่อม เป็นเส้นเชื่อมมีทิศทาง (edges) พร้อมระยะทาง เวลาเดิน ทิศเลี้ยว และ ธงว่ารถเข็นผ่านได้หรือไม่

ผลคือการหาเส้นทางกลายเป็นปัญหา shortest path ธรรมดา และการรองรับรถเข็น กลายเป็นแค่การตัด edge ที่ `is_accessible = false` ออกจากกราฟ โดยไม่ต้องเขียนตรรกะพิเศษแยก

3.2 แผนภาพความสัมพันธ์ (ER)



3.3 ตารางทั้ง 17 ตาราง

#	ตาราง	หน้าที่	คีย์สำคัญ
1	buildings	อาคาร	code UNIQUE
2	floors	ชั้นในอาคาร	UNIQUE (building_id, level)
3	departments	แผนก · สีประจำแผนก · จุดทางเข้า	code UNIQUE
4	nodes	ทุกจุดในผัง — ห้อง ลิฟต์ ทางแยก ประตู	code, qr_token UNIQUE
5	edges	เส้นเชื่อมระหว่างจุด มีทิศทาง	UNIQUE (from, to)
6	node_photos	รูปถ่ายจริงของแต่ละจุด	FK → nodes
7	service_hours	เวลาทำการของแผนกหรือจุด	FK → departments/nodes
8	pathway_templates	แม่แบบการมา เช่น เบาหวานตามนัด	code UNIQUE
9	template_steps	ขั้นตอนในแม่แบบ พร้อมกำหนดเวลา	UNIQUE (template_id, sequence)
10	template_step_prereqs	ขั้นไหนต้องทำหลังขั้นไหน (N:N วนตัวเอง)	PK คู่
11	users	เจ้าหน้าที่ · bcrypt · บทบาท · แผนกที่ดูแล	username UNIQUE
12	patients	ผู้ป่วย — เก็บแค่ HN ชื่อเรียก ภาษา	hn UNIQUE
13	visits	การมาหนึ่งครั้ง · token · ตำแหน่งล่าสุด	token, short_code UNIQUE
14	visit_steps	ขั้นตอนจริงของ visit นั้น · สถานะ · เวลา	FK → visits ON DELETE CASCADE
15	visit_step_prereqs	เงื่อนไขลำดับของ visit นั้น	PK คู่
16	notifications	ข้อความแจ้งผู้ป่วย สองภาษา	FK → visits
17	audit_logs	ใครทำอะไรกับอะไรเมื่อไร · แก้ไม่ได้	append-only

3.4 การตัดสินใจที่ควรอธิบาย

เรื่อง	ทำไมออกแบบแบบนี้
แยก <code>template_steps</code> ออกจาก <code>visit_steps</code>	ตอนเปิด <code>visit</code> ระบบ คัดลอก ขั้นตอนจากแม่แบบมาเป็นของ <code>visit</code> นั้น ถ้าผู้ดูแลแม่แบบ ตอนบ่าย ผู้ป่วยที่เปิดบัตรตอนเช้าต้องไม่ถูกเปลี่ยนแผนกลางคัน
<code>prereqs</code> เป็น N:N วนตัวเอง ไม่ใช่ คอลัมน์ <code>after_step</code>	ขั้นหนึ่งอาจต้องรอหลายขั้น เช่น "พบแพทย์" ต้องรอทั้งเจาะเลือดและเอกซเรย์ ตารางเชื่อมโยงจึงเป็นรูปเดียวที่ไม่บังคับให้ลำดับเป็นเส้นตรง
<code>visit_steps.service_point_id</code> ชี้ไป <code>nodes</code> ไม่ <code>snapshot</code> ชื่อห้อง	ถ้าห้องย้าย ผู้ป่วยที่ยังเดินอยู่ต้องถูกพาไปห้องใหม่ ไม่ใช่ห้องเก่าที่จอดไว้
<code>edges</code> มีทิศทาง	บันไดหนีไฟเดินลงได้อย่างเดียว · ทางเดินที่เลียวย้ายขามา ต้องเลียวขากลับ ตอนนำเข้า CSV ที่มี <code>bidirectional=1</code> ระบบสร้างสองแถวและสลับซ้ายขวาให้เอง
<code>visits.token</code> 32 ตัวอักษร + <code>short_code</code> 8 ตัว	token อยู่ใน QR สำหรับสแกน · <code>short_code</code> พิมพ์ได้ QR สำหรับคนที่กล้องเสียหรือบัตรเปียก ใช้ตัวอักษรที่อ่านไม่สับสน (ตัด I O 0 1 ออก)
ไม่เก็บเลขบัตรประชาชน	ระบบไม่ต้องใช้ จึงไม่เก็บ — ลดความเสียหายถ้าข้อมูลรั่ว

4. การนอร์มัลไลเซชัน

ฐานข้อมูลอยู่ในรูป 3NF ทั้งหมด ตรวจสอบทีละระดับ

4.1 1NF — ทุกช่องเก็บค่าเดียว

ไม่มีคอลัมน์ไหนเก็บรายการคั่นด้วยจุลภาค ตัวอย่างที่ล่อใจที่สุดคือ "ขั้นนี้ต้องรอขั้นไหนบ้าง" ซึ่งถ้าทำง่าย ๆ จะกลายเป็น `prereq = "1;3;4"` — เราแยกเป็นตาราง `visit_step_prereqs` หนึ่งแถวต่อหนึ่งเงื่อนไขแทน

หมายเหตุ: `visits.profile` เป็น JSONB เก็บธงความต้องการพิเศษ ซึ่งเป็นการยกเว้นที่ตั้งใจ — ธงเหล่านี้ไม่มีความสัมพันธ์กับ ตารางอื่นและไม่เคยถูก query แยกรายธง ระบบตรวจค่าที่รับเข้าให้เหลือเฉพาะธงที่รู้จัก 4 ตัวก่อนบันทึกเสมอ

4.2 2NF — ไม่มีคอลัมน์ที่ขึ้นกับคีย์แค่บางส่วน

ตารางที่มีคีย์ประกอบคือตาราง `prereqs` ทั้งสองตัว ซึ่งมีแค่สองคอลัมน์ที่เป็นคีย์ทั้งคู่ จึงไม่มีคอลัมน์อื่นให้ขึ้นกับคีย์บางส่วนได้

4.3 3NF — ไม่มีคอลัมน์ที่ขึ้นกับคอลัมน์ที่ไม่ใช่คีย์

จุดที่ต้องระวังคือโซ่ อาคาร → ชั้น → แผนก → จุด ถ้าออกแบบแบบง่าย ตาราง `nodes` จะมีคอลัมน์ `building` และ `floor` อยู่ในตัวเอง ซึ่งทำให้ชื่ออาคารถูกเก็บซ้ำ 50 ครั้ง และถ้าเปลี่ยนชื่ออาคารต้องไล่แก้ทุกแถว

เราแยกเป็นสามตารางแทน `nodes` เก็บแค่ `floor_id` ส่วนชั้นรู้ว่าตัวเองอยู่อาคารไหน ผลคือ

- เปลี่ยนชื่ออาคารแก้แถวเดียว มีผลทุกหน้าจอทันที
- ไม่มีทางเกิดสภาพที่จุดสองจุดบอกว่าอยู่ชั้น 3 อาคาร B แต่คนละแผนกกันแบบขัดแย้ง

ตัวอย่างการสืบย้อนในของจริง

```
SELECT n.code, n.name_th, d.code AS dept, f.level AS floor, b.code AS building
FROM nodes n
JOIN floors f      ON f.id = n.floor_id
JOIN buildings b  ON b.id = f.building_id
LEFT JOIN departments d ON d.id = n.department_id
WHERE n.is_active;
```

ทุกหน้าจอที่ต้องแสดง "อาคาร B ชั้น 3" ใช้ join ชุดนี้ ไม่มีที่ไหนเก็บสตริงสำเร็จรูปไว้

4.4 ที่ตั้งใจไม่ normalize

จุด	เหตุผล
visit_steps คัดลอก name_th, deadline_time, est_minutes จากแม่แบบ	เป็น snapshot ที่ตั้งใจ ไม่ใช่ข้อมูลเข้าซ้อน — บัตรคิวที่ออกไปแล้วต้องไม่เปลี่ยนเนื้อหาเมื่อแม่แบบถูกแก้
audit_logs.detail เป็น JSONB	แต่ละเหตุการณ์มีรายละเอียดคนละครูป การบังคับให้เป็นคอลัมน์ตายตัวจะได้ตารางที่มีคอลัมน์ว่างเป็นส่วนใหญ่

5. API Contract

ทุกปลายทางอยู่ใต้ `/api` · เจ้าหน้าที่ใช้ JWT ใน `Authorization: Bearer` · ผู้ป่วยใช้ token ที่อยู่ในลิงก์ของบัตรคิว ไม่
ต้องเข้าสู่ระบบ

5.1 รายการทั้งหมด 28 ปลายทาง

เมธอด	ปลายทาง	ใครเรียกได้
GET	/api/health	เปิด
POST	/api/auth/login	เปิด
GET	/api/auth/me	ต้องเข้าสู่ระบบ
GET	/api/map	เปิด
GET	/api/nodes	เปิด
GET	/api/nodes/by-qr/:token	เปิด
GET	/api/templates	เปิด
GET	/api/patients	registrar · admin
POST	/api/patients	registrar · admin
POST	/api/visits	registrar · admin
GET	/api/visits	registrar · admin · station · executive
GET	/api/visits/:id	registrar · admin · station
POST	/api/visits/:id/steps	station · admin
GET	/api/stations	station · admin · registrar · executive
GET	/api/stations/:code/queue	station · admin
POST	/api/steps/:id/status	station · admin · registrar
GET	/api/p/code/:code	เปิด (แลกรหัส 7 ตัวเป็น token)
GET	/api/p/:token	เปิด (token คือสิทธิ์)
POST	/api/p/:token/position	เปิด
POST	/api/p/:token/arrive	เปิด
POST	/api/p/:token/notifications/read	เปิด
POST	/api/p/:token/demo/:action	เปิด (เฉพาะโหมดเดโม)
PATCH	/api/edges/:id	admin
GET	/api/edges/:id/impact	admin
GET	/api/audit	admin
GET	/api/demo/clock	เปิด

POST	<code>/api/demo/clock</code>	เปิด (เฉพาะโหมดเดโม)
POST	<code>/api/demo/reset</code>	เปิด (เฉพาะโหมดเดโม)

5.2 รูปแบบคำตอบที่ตกลงกันได้

ทุกปลายทางที่เปลี่ยนสถานะของ visit คือ สถานะเต็มชุดเดียวกันเสมอ หน้าจอจึงไม่ต้องเดาผลลัพธ์เองและไม่ต้องยิงซ้ำเพื่อ refresh

```
{
  "visit": { ticket_no, short_code, token, status, lang, profile, patient_name,
    template: { code, name_th, name_en },
    at: { code, name_th, building, floor, source, confirmed_at } },
  "steps": [ { id, sequence, name_th, status, point: {...},
    deadline_time, requires: [id], arrived_at, called_at, finished_at } ],
  "next_step_id": 12,
  "ready_step_ids": [12, 15],
  "route": { to, seconds, distance_m,
    legs: [ { act, turn, m, to, arrive, side, building, floor } ],
    instructions: [ { n, arrow, icon, distance_m, text_th, text_en } ] },
  "warnings": [ { code: "deadline_soon", step_id, minutes_left } ],
  "notifications": [ { id, message_th, message_en, created_at, read_at } ],
  "server_time": "2569-09-20T10:40:00.000Z"
}
```

5.3 รูปแบบข้อผิดพลาด

```
{ "error": { "code": "CONFLICT",
  "message_th": "ชั้นนี้อยู่สถานะ \"pending\" เปลี่ยนเป็น \"done\" ไม่ได้",
  "message_en": "Cannot change a step from \"pending\" to \"done\"" } }
```

ข้อความมาสองภาษาเสมอ หน้าจอเลือกใช้ตามภาษาที่ผู้ใช้เลือก ไม่ต้องแปลเอง รหัสที่ใช้จริง — `BAD_REQUEST` 400 · `UNAUTHORIZED` 401 · `FORBIDDEN` 403 · `NOT_FOUND` 404 · `CONFLICT` 409 · `EXPIRED` 410 · `PREREQ_CYCLE` 422 · `INTERNAL` 500

บทเรียนจากวันที่ 2 — ตอนย้ายไป NestJS ทุก `@Post` เริ่มตอบ 201 แทน 200 เพราะเป็นค่าปรีายของเฟรมเวิร์ก ซึ่งทำให้ contract ที่ตกลงไว้เปลี่ยนโดยไม่ได้ตั้งใจ ต้องใส่ `@HttpCode(200)` กำกับ 8 จุด ยกเว้น `POST /patients` และ `POST /visits` ที่สร้างทรัพยากรใหม่จริงจึงคง 201 ไว้

6. อัลกอริทึมที่เป็นหัวใจ

6.1 หาเส้นทาง — Dijkstra ที่รู้จักความต้องการพิเศษ

ต้นทุนของแต่ละ edge ไม่ใช่ระยะทางคงที่ แต่คิดจากโปรไฟล์ผู้ป่วย

```
function costFor(profile) {
  const slower = (profile.elderly || profile.wheelchair) ? 1.5 : 1;

  return (edge) => {
    if (profile.wheelchair && !edge.is_accessible) return Infinity; // ตัดออกจากกราฟ

    let seconds = edge.walk_seconds * slower;
    if (profile.elderly && edge.kind === 'stairs') seconds += 60; // บันไดคือการจะจริง
    if (profile.has_phone === false && edge.turn && edge.turn !== 'straight') seconds += 5;
    return seconds; // ไม่มีมือถือ = ทุกทางเลี้ยวเสี่ยงหลง
  };
}
```

คิ่ณค่า `Infinity` แทนการลบ edge ออกจริง ทำให้ใช้กราฟชุดเดียวกับทุกคนได้ โดยไม่ต้องสร้างกราฟใหม่ต่อผู้ป่วยหนึ่งราย

ผลที่พิสูจน์ด้วยเทสต์

สถานการณ์	ผลที่ระบบเลือก
คนเดินปกติ จากทางเข้าไปห้องแล็บ	เลือกบันได 44 วินาที เพราะเร็วกว่าลิฟต์
ผู้ใช้รถเข็น เส้นทางเดียวกัน	เลี่ยงบันได ไปทางลิฟต์ ใช้เวลามากกว่า
ผู้สูงอายุ เส้นทางเดียวกัน	เส้นทางเดิมแต่เวลามากขึ้นตามตัวคูณ
ปิดลิฟต์แล้วผู้ใช้รถเข็นไปไม่ถึง	คืน <code>null</code> และขึ้นค่าเตือน ไม่ใช่เส้นทางที่เดินไม่ได้

6.2 เลือกชั้นถัดไป — ไม่ใช่แค่เรียงตามลำดับ

ชั้นถัดไปไม่ได้เลือกจากเลขลำดับอย่างเดียว แต่จากสามเงื่อนไขซ้อนกัน

- ทำได้ตอนนี้ไหม — ชั้นที่ต้องทำก่อนต้องเสร็จหรือถูกข้ามหมดแล้ว
- มีกำหนดเวลาที่กำลังจะไม่ทัน — ถ้าเดินไปถึงแล้วทำเสร็จเกิน (กำหนด - 20 นาที) ระบบยกชั้นนั้นขึ้นมาก่อนแม้จะเดินไกลกว่า
- ถ้าไม่มีอะไรเร่ง ก็เลือกที่เดินใกล้ที่สุด — ลำดับเดิมเป็นตัวตัดสินเมื่อเสมอกัน

ตัวอย่างจริง: ผู้ป่วยเบาหวานต้องเจาะเลือดก่อน 11:00 น. ถ้าเปิดบัตร 10:40 น. ระบบจะพาไปเจาะเลือดก่อน แม้ว่าจุดคัดกรองจะอยู่ไกลกว่า และแสดงเหตุผลให้ผู้ป่วยเห็นบนหน้าจอ

6.3 แทรกชั้นกลางวัน — Topological Sort (Kahn)

เมื่อแพทย์สั่งตรวจเพิ่ม ชั้นใหม่ต้องไปอยู่ตำแหน่งที่ถูกต้องโดยไม่มีใครระบุเอง

1. แทรกชั้นใหม่ด้วยลำดับกึ่งกลาง (ก่อน "ชำระเงิน" ลำดับ 6 → ได้ 5.5)
2. บันทึกเงื่อนไข: เอกซเรย์ต้องทำหลัง "พบแพทย์"
3. เรียงใหม่ทั้งแผนด้วย Kahn's algorithm (ลำดับเดิมเป็นตัวตัดสินเมื่อเลือกได้หลายชั้น)
4. ถ้าเรียงไม่ครบ = เงื่อนไขวนกลับมาหาตัวเอง → ปฏิเสธด้วย 422 PREREQ_CYCLE
5. เขียนลำดับ 1..n กลับลงฐานข้อมูลในทรานแซกชันเดียวกัน

ผลจริงที่วันนี้ — เอกซเรย์ถูกวางเป็นลำดับที่ 6 หลัง พบแพทย์ (ลำดับ 5) และ ก่อน ชำระเงิน (ลำดับ 7) โดยไม่มีใครบอกตำแหน่ง

6.4 ประโยคนำทาง — ข้อมูลล้วน ไม่ใช่ข้อความสำเร็จรูป

ระบบไม่เก็บประโยค "เลี้ยวขวาแล้วเดิน 24 เมตร" ไว้ในฐานข้อมูล แต่เก็บเป็นข้อมูล { turn: 'right', m: 24, to: 'LIFT-B' } แล้วประกอบเป็นประโยคตอนแสดงผลตามภาษาที่เลือก การเพิ่มภาษาที่ 6 จึงเป็นการเพิ่มชุดคำแปล ไม่ต้องแตะข้อมูลเส้นทางเลย

ทางเดินตรงที่ต่อเนื่องกันถูกยุบเป็นข้อเดียวและรวมระยะทาง เพื่อไม่ให้ผู้ป่วยต้องอ่าน 12 ข้อ สำหรับทางเดินเส้นเดียว

6.5 ผลกระทบก่อนปิดเส้นทาง

ก่อนผู้ดูแลระบบกดปิดลิฟต์ ระบบคำนวณให้ดูก่อนว่ากระทบใครบ้าง โดย จำลองการปิดบนกราฟสำเนา ไม่แตะฐานข้อมูล แล้วเทียบจำนวนจุดที่ไปถึงได้ และเวลารวมที่ต้องใช้ ระหว่างคนเดินปกติกับผู้ใช้รถเข็น

7. การออกแบบ UX/UI

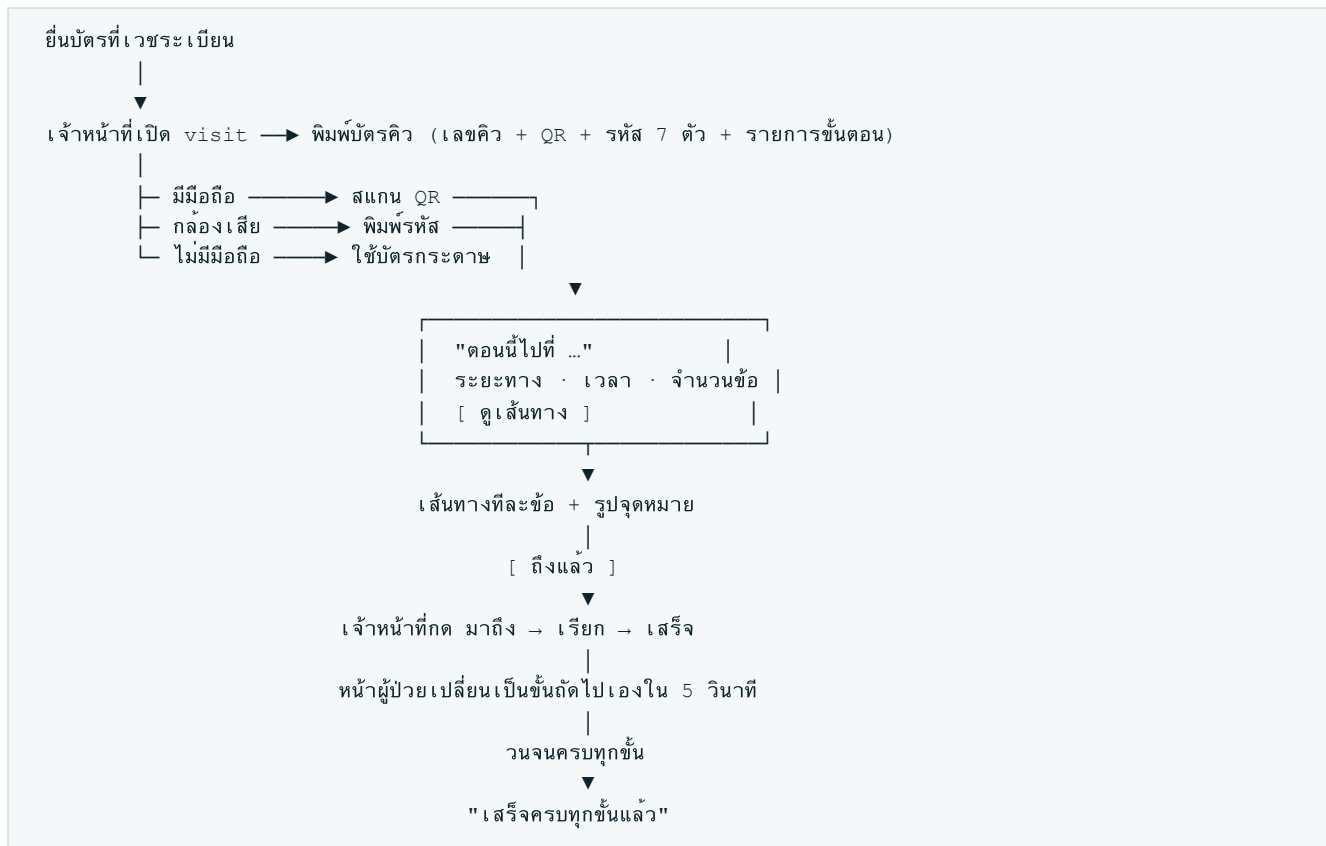
7.1 หลักที่ยึด

หลัก	ทำอะไร
ผู้ป่วยเห็นคำตอบเดียวต่อหนึ่งครั้ง	หน้าแรกคือ "ตอนนี้ไปที่ไหน" ตัวใหญ่ ไม่ใช่รายการทั้งวัน — รายการอยู่ถัดลงไป
เจ้าหน้าที่กดปุ่มเดียว	มาถึง → เรียก → เสร็จ ระบบบันทึกเวลาให้เอง ไม่มีฟอร์ม
คนที่อ่านไม่ออกต้องยังใช้ได้	ลูกศร ไอคอน และตัวเลขเหมือนกันทุกภาษา เปลี่ยนภาษาแล้วเดินต่อได้ทันที
คนหลงต้องเลือกจากสิ่งที่ "มองเห็น"	หน้า "ฉันอยู่ที่ไหน" เป็นตารางรูปของแต่ละจุด ไม่ใช่รายชื่อห้อง
สายตาเลื่อนรางต้องอ่านได้	ปุ่ม + ขยายทั้งหน้าด้วยตัวคูณเดียว ปุ่มโตตามไม่เสียพื้นที่กด

7.2 หน้าจอทั้งหมด

หน้า	ชนิด	ผู้ใช้	หน้าที่
/login	React	ทุกคน	เข้าสู่ระบบ · ทางเข้าผู้ป่วยด้วยรหัส QR
/registrar	React	เวชระเบียน	สามขั้น: ค้นผู้ป่วย → เลือกแม่แบบ → ตี๊กความต้องการพิเศษ แล้วพิมพ์บัตร A5
/station	React	จุดบริการ	คิวของแผนก · ปุ่มเปลี่ยนสถานะ · ส่งตรวจเพิ่ม
/patient	React	ผู้ป่วย	ชั้นถัดไป · เส้นทางที่ละเอียด · ภาพรวมทั้งวัน · 5 ภาษา
/patient-map.html	นิ่ง	ผู้ป่วย	แผนที่ที่ละเอียด ชูตามช่วงเดิน
/privacy.html	นิ่ง	ผู้ป่วย	ประกาศความเป็นส่วนตัว 5 ภาษา
/helpdesk.html	นิ่ง	ประชาสัมพันธ์	ค้นด้วยเลขคิว · พิมพ์ไปเส้นทางให้
/admin.html	นิ่ง	ผู้ดูแลระบบ	หน้าแกระบบ · ตรวจผังอัตโนมัติ
/impact.html	นิ่ง	ผู้ดูแลระบบ	ผลกระทบก่อนปิดเส้นทาง
/map.html	นิ่ง	ผู้ดูแลระบบ	ผังรวมทุกอาคาร 3 มุมมอง
/pathway.html	นิ่ง	ผู้ดูแลระบบ	แผนภาพแม่แบบการมาหนึ่งครั้ง

7.3 User Flow ของผู้ป่วยหนึ่งคน



8. ความปลอดภัยและความเป็นส่วนตัว

เรื่อง	วิธีทำ
รหัสผ่าน	bcrypt rounds 10 · หน้าเว็บไม่เคยเห็นรหัสที่ถูกต้อง · ข้อความผิดพลาดเหมือนกันทั้งชื่อผิดและรหัสผิด
สิทธิ์	ตรวจที่ API ทุกคำขอด้วย AuthGuard + @Roles() · การซ่อนปุ่มบนหน้าจอไม่ถือเป็นการป้องกัน · อ่านผู้ใช้จากฐานข้อมูลใหม่ทุกครั้ง เพื่อถูกปิดใช้งานระหว่างวัน
SQL Injection	ทุก query เป็น parameterized · มีสคริปต์ npm run lint:sql ตรวจว่าไม่มี template literal ที่หน้าตาเป็น SQL แล้วมี \${...} อยู่ข้างใน
token ของผู้ป่วย	สุ่ม 16 ไบต์ · หมดยอายุสิ้นวัน · เลยแล้วลิงก์เดิมเปิดไม่ได้ ตอบ 410
ข้อมูลที่ไม่เก็บ	ไม่เก็บเลขบัตรประชาชน · หน้าจอสาธารณะไม่แสดง HN เต็ม · บัตรคิวไม่มีเลขประจำตัวผู้ป่วย
audit	append-only จরিระดับฐานข้อมูล — REVOKE UPDATE, DELETE ON audit_logs · เวลาใช้ของเซิร์ฟเวอร์เสมอ ไม่รับจาก client · เจ้าหน้าที่ที่เปิดดู visit ของใครก็ถูกบันทึก
ความยินยอม	หน้าประกาศความเป็นส่วนตัว 5 ภาษา · เวชระเบียนดีกว่าแจ้งแล้ว ระบบบันทึกว่าใครแจ้งเมื่อไร

9. ขั้นตอนการพัฒนา — วันที่ 1 (เสาร์ 19 กันยายน)

Problem & Design Sprint

9.1 ช่วงกิจกรรม 09.00–16.00 น.

เวลา	Sprint	ผลที่ได้
09.20–10.00	Problem Briefing	รับโจทย์จริงจากผู้ให้โจทย์ WIL และเงื่อนไขทางธุรกิจ
10.00–10.30	จัดทีม เลือกโจทย์	ทีม 3 คน · แบ่งบทบาท · ถาม-ตอบกับผู้ให้โจทย์
10.45–12.00	Sprint 1 — Requirement & Scope	User Story · Use Case · ขอบเขต MVP (หัวข้อ 1 ของเอกสารนี้)
13.00–14.30	Sprint 2 — System & Database Design	สถาปัตยกรรม · ER · Normalization · API Contract (หัวข้อ 2–5)
14.45–15.45	Sprint 3 — UX/UI & Prototype	แบบหน้าจอ 11 หน้า · User Flow (หัวข้อ 7)
15.45–16.00	สรุปและมอบหมายงานต่อเนื่อง	แผนลงมือของเย็นวันนั้น

9.2 งานต่อเนื่องเย็นวันที่ 1 — 19.28 ถึง 23.52 น. (27 commit)

ทีมใช้ช่วง "งานต่อเนื่อง" ลงมือสร้างจริง ทำให้เช้าวันที่ 2 เริ่มจากระบบที่ต่อครบสามชั้นแล้ว

ช่วงเวลา	สิ่งที่สร้าง
19.28–19.36	วาง docker compose (api + db + web) · <code>schema.sql</code> 17 ตาราง 3NF · ข้อมูลตัวอย่าง CSV · แบบหน้าจอและหน้าอธิบายการออกแบบข้อมูล
19.47–20.35	ต้นแบบหน้าผู้ป่วยที่กดได้จริงบนมือถือ · รองรับ 5 ภาษา · หน้าเวชระเบียน · ภาพประกอบของทุกจุดในผัง · เลือกตำแหน่งจากรูปแทนรายชื่อ
21.02–22.02	ผังเส้นทางที่ วาดจากข้อมูลจริง ไม่ใช่รูปที่แก้ด้วยมือ · แผนภาพขั้นตอนหนึ่ง visit · แผนที่ฉบับผู้ป่วย
22.16–22.54	หน้าผลกระทบเมื่อปิดเส้นทาง · หน้าแรกของผู้ดูแลระบบ · จุดบริการผูกกับหน้าผู้ป่วยจริง · โต๊ะช่วยเหลือ
23.10–23.52	ประกาศความเป็นส่วนตัว + บันทึกการแจ้ง · เอกสารระบบตามที่เราสร้างจริง · สไลด์นำเสนอ 14 หน้า · ชุดทดสอบอัตโนมัติ

9.3 Stack เมื่อจบวันที่ 1

```

เบราว์เซอร์ → nginx :5173 — ไฟล์ HTML/CSS/JS ล้วน ไม่มี build step
└─ /api/ → Express 5 :3000 → PostgreSQL

```

10. ขั้นตอนการพัฒนา — วันที่ 2 (อาทิตย์ 20 กันยายน)

Development & Presentation

10.1 สิ่งที่น่าสนใจทำและทำไม

Sprint 4 และ 5 ของวันที่ 2 คือ Backend Implementation และ Frontend Implementation ทีมเลือกใช้ช่วงนี้ **ยกระดับ stack** ทั้งสองฝั่งเป็นเฟรมเวิร์กมาตรฐานอุตสาหกรรม แทนที่จะเพิ่มฟีเจอรใหม่ เพราะ

- ฟีเจอรตามโจทย์ครบตั้งแต่เย็นวันที่ 1 แล้ว การเพิ่มของใหม่มีแต่จะเพิ่มความเสี่ยง
- Express ที่เขียนเองไม่มีโครงสร้างสิทธิ์ที่ตรวจสอบได้ชัด · NestJS ให้ guard และ decorator ที่อ่านแล้วรู้ทันทีว่าใครเรียกได้
- HTML ล้วนทำให้โทเคนการออกแบบถูกคัดลอกซ้ำในหัวไฟล์ทุกใบ แก๊สเดียวต้องไล่แก้ 11 ไฟล์
- TypeScript จับความผิดพลาดของรูปข้อมูลระหว่าง front กับ back ได้ตั้งแต่ตอนคอมไพล์

เงื่อนไขที่ตั้งไว้ก่อนเริ่ม — ฐานข้อมูลห้ามแตะ · API contract ห้ามเปลี่ยน · ลิงก์เดิมที่พิมพ์ลงบัตรคิวไปแล้วต้องยังใช้ได้ · ถ้าทำไม่เสร็จก่อนพักเที่ยงให้ย้อนกลับทั้งหมด

10.2 Sprint 4 — Backend (Express 5 → NestJS 11 + TypeScript)

จาก	เป็น
ไฟล์ route 7 ไฟล์	controller 7 ตัว ครอบคลุม 28 ปลายทาง
<code>requireRole()</code> เป็น middleware ต่อกันเอง	<code>AuthGuard + @Roles()</code> อ่านแล้วรู้ทันทีว่าใครเรียกได้
error handler กลางเขียนเอง	<code>ApiExceptionHandler</code> คงรูป envelope เดิมทุกตัวอักษร
<code>pg.Pool</code> เป็นตัวแปรระดับโมดูล	<code>DbService</code> ที่ฉีดเข้าไปได้ · SQL และ schema เดิมไม่เปลี่ยน
ตรรกะโดเมนปนกับ route	ฟังก์ชันล้วนแยกออกมา ทดสอบได้โดยไม่ต้องมีฐานข้อมูล
เทสต์ด้วย <code>node --test</code>	jest · 24 กรณีผ่านทั้งหมด

10.3 Sprint 5 — Frontend (HTML + nginx → Next.js 15 + TypeScript)

- หน้าต่อ API จริง 4 หน้าเขียนใหม่เป็น React — `/login /registrar /station /patient`
- อีก 7 หน้ายังเป็น HTML นิ่งที่สร้างจากข้อมูลฝั่ง ซึ่งพอสำหรับภาพนิ่งที่ไม่เปลี่ยนระหว่างวัน
- โทเคนการออกแบบที่เคยซ้ำในทุกไฟล์ รวมเหลือ `globals.css` ที่เดียว
- ตัวสร้าง QR เล็กพิ่ง CDN ย้ายมาใช้แพ็คเกจจาก npm — ทำงานได้แม้ไม่มีเน็ต
- ลิงก์เดิมนยังใช้ได้ — `/patient-mobile.html?t=...` redirect ไป `/patient?t=...`

10.4 ปัญหาที่เจอระหว่างย้ายและสาเหตุจริง

#	อาการ	สาเหตุและวิธีแก้
1	เข้าสู่ระบบไม่ได้ ขึ้น Internal Server Error	Next.js แปลง <code>rewrites()</code> เป็นไฟล์ตั้งแต่ตอน build ค่า <code>localhost:3000</code> ที่ถูกฝังไว้จึงหมายถึงตัว container เอง ไม่ใช่ service ของ API · การตั้ง environment ตอน runtime ไม่มีผล ต้องส่งเป็น build argument
2	ทุก POST ตอบ 201	ค่าปรีายของ NestJS · ใส่ <code>@HttpCode(200)</code> 8 จุดให้ตรง contract เดิม
3	<code>/mockups/</code> ตอบ 404	Next ไม่เสิร์ฟ <code>index.html</code> ของโพลเดอร์แบบที่ nginx ทำให้ · เพิ่ม rewrite
4	แผนที่ผู้ป่วยว่างเปล่า	เมื่อผู้ป่วยยืนอยู่ที่จุดหมายแล้ว เส้นทางเป็นอาเรย์ว่าง ซึ่งไม่ใช่ <code>null</code> ตัวตรวจเดิมจับไม่ได้จึงพังกลางการวาด · เพิ่มการตรวจ และแยกข้อความ "อยู่ที่หมายแล้ว" ออกจาก "ไปไม่ถึง" เพราะบอกผู้ใช้รถเห็นว่าไม่มีเส้นทางทั้งที่เดินมาถึงแล้ว เป็นข้อมูลที่ผิด
5	เอกสารสั่งคำสั่งที่รันไม่ได้	อิมเมจของ API ตัด devDependencies ออก จึงไม่มี jest ข้างใน · ย้ายไปรันบนเครื่อง

10.5 ผลการทดสอบกับฐานข้อมูลจริง

สิ่งที่ทดสอบ	ผล
เข้าสู่ระบบครบ 5 บัญชี · รหัสผิด	200 ทุกบัญชี · รหัสผิดได้ 401
สิทธิ์ข้ามแผนก — lab1 ขอคิวแผนกอื่น	403 ตามที่ควร
กฎลำดับสถานะ — <code>pending</code> → <code>done</code> ตรง ๆ	409 CONFLICT
เส้นทางโปรไฟล์รถเข็น	147 เมตร · 9 คำสั่ง · ประโยคไทย ถูก
แทรกชั้นเอกซเรย์	เข้าลำดับ 6 ถูกตำแหน่ง
กด "เสร็จ" ที่จุดบริการ	หน้าผู้ป่วยเปลี่ยนตาม + แจ้งเตือน
เส้นทางเว็บ 16 เส้น + redirect เดิม 4 เส้น	ครบ

10.6 Stack เมื่อจบวันที่ 2

เบราร์เวอร์ → Next.js 15 :5173	React 4 หน้า + HTML นิ่ง 7 หน้า
└─ /api/ → NestJS 11 :3000	→ PostgreSQL 16

ฐานข้อมูลเป็นชั้นเดียวที่ไม่เปลี่ยนแปลงทั้งสองวัน — `schema.sql` ที่เขียนเย็นวันที่ 1 ยังเป็นไฟล์เดิมทุกบรรทัด ซึ่งเป็นหลักฐานว่าการออกแบบฐานข้อมูลตั้งแต่ต้นถูกต้องพอที่จะรองรับ การเปลี่ยนเฟรมเวิร์กทั้งสองฝั่งได้โดยไม่ต้องแก้ตาม

11. การทดสอบ

11.1 ชุดทดสอบอัตโนมัติ 24 กรณี

รันด้วย `npx jest` ในโพลเดอร์ `backend/` ใช้เวลาประมาณ 9 วินาที ทดสอบตรรกะล้วนโดยไม่ต้องมีฐานข้อมูล

ชุด	จำนวน	ครอบคลุมอะไร
graph.test.ts	7	Dijkstra ตามโปรไฟล์ · รถเข็นเลี้ยงบันได · ทางเดียว · จุดที่ไม่ถึง
pathway.test.ts	8	เงื่อนไขลำดับ · การเลือกชั้นถัดไป · กำหนดเวลา · topological sort · การตรวจวงจร
legs.test.ts	4	ยุบทางเดินตรง · ทิศขึ้นลงลิฟต์ · ข้อสุดท้ายบอกด้านที่จุดหมายอยู่
csv.test.ts	5	อ่านไฟล์นำเข้า · เครื่องหมายคำพูดซ้อน · บรรทัดว่าง

11.2 การตรวจอัตโนมัติ

- `npm run lint:sql` — สแกนโค้ดทั้งหมดหา template literal ที่หน้าตาเป็นคำสั่ง SQL แล้วมีการแทรกค่าด้วย `{...}` ซึ่งเป็นช่องทาง SQL injection · ปัจจุบันผ่าน ไม่มีจุดใดผิด
- `npx tsc --noEmit` — ตรวจชนิดข้อมูลทั้งสองฝั่งก่อน build

11.3 การทดสอบด้วยมือ

ทดสอบเส้นทางสาธิตทั้งเส้นหลัง reset ข้อมูลทุกครั้งที่แก้โค้ด — เปิด visit → สแกน QR → เปลี่ยนตำแหน่ง → เจ้าหน้าที่กดสามปุ่ม → แทรกชั้น → ปิดลิฟต์

12. ข้อจำกัดและงานต่อ

12.1 ที่รู้ตัวว่ายังไม่ได้ทำ

เรื่อง	สถานะ
ยกระดับขึ้นใช้งานบนอินเทอร์เน็ตสาธารณะ	วิเคราะห์แล้ว ยังไม่ทำ (ดู 12.2)
หน้าจัดการผู้ใช้และสิทธิ์	ยังไม่มี
หน้าดู audit log	มี API แล้ว ยังไม่มีหน้าจอ
นำเข้า CSV ผ่านหน้าเว็บ	มี API แล้ว ยังใช้ผ่านคำสั่ง
ให้หน้าฝั่ง ผลกระทบ ได้ะช่วยเหลือ ดึงข้อมูลจาก API แทนไฟล์ที่ build ไว้	ใช้ได้ แต่เป็นภาพนิ่ง
backup / restore	ออกแบบไว้ ยังไม่ได้ทำ

12.2 ทำไมยังไม่ขึ้น Cloudflare Workers

หน้าเว็บขึ้นได้ทันที เพราะทั้ง 4 หน้าเป็น client component ไม่มี server action แต่ API ขึ้นตรง ๆ ไม่ได้ ติด 6 จุด

#	จุด	ปัญหา
1	NestJS HTTP adapter	ผูกกับ Express ซึ่ง Workers ไม่มี · ต้องเปลี่ยนเป็น Hono
2	bcrypt rounds 10	ใช้ CPU 60–100 ms เกินโควตา 10 ms ของแผนฟรี
3	<code>pg.Pool</code>	ต้องต่อผ่าน Hyperdrive และใช้ Postgres ภายนอก
4	<code>fs.readFile</code>	อ่าน schema และ CSV ตอน reset · Workers ไม่มีระบบไฟล์
5	แคชกราฟผัง	เก็บในหน่วยความจำของ instance ไม่แชร์ข้าม isolate
6	นาฬิกาจำลอง	เหตุผลเดียวกับข้อ 5 · ตั้งแล้วเครื่องอื่นไม่เห็น

ทีมเลือกลงทุนเวลากับความครบและความถูกต้องของระบบก่อน แล้วสาธิตจากเครื่องผ่านเครือข่ายภายใน ซึ่งพิสูจน์การทำงานจริงได้ครบเท่ากัน

12.3 ถ้ามีเวลาต่อ

1. เขียน API ใหม่เป็น Hono แล้วขึ้น Workers พร้อม Postgres บน Neon ผ่าน Hyperdrive
2. ให้นำที่ยังเป็น HTML นึงดึงจาก `GET /api/map` เพื่อให้ผังเปลี่ยนแล้วเห็นทันทีทุกหน้า
3. นำผังจริงของโรงพยาบาลเข้าระบบผ่าน CSV แล้ววัดผลกับผู้ป่วยจริง
4. เพิ่มหน้าจัดการผู้ใช้ หน้าดู audit และระบบ backup

สรุป — CarePath ต่อครบทั้งสามขั้นและทำงานกับข้อมูลจริงในฐานข้อมูล การออกแบบที่เก็บผังโรงพยาบาลเป็นกราฟทำให้การรองรับผู้ใช้รถเข็น การคิดเส้นทางใหม่เมื่อลิฟต์ปิด และการแทรกชั้นตอนกลางวัน เป็นผลพลอยได้ของโครงสร้างข้อมูล ไม่ใช่ฟีเจอร์ที่ต้องเขียนแยกทีละอย่าง และการที่ `schema.sql` ไม่ต้องแก้เลยแม้เปลี่ยนเฟรมเวิร์กทั้งสองฝั่งในวันที่ 2 เป็นหลักฐานว่าการออกแบบฐานข้อมูลในวันที่ 1 วางไว้ถูกต้อง